FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	John Louis Panganiban Julliane Gabrielle Magbanua
Section:	AN41
Professor:	Ms. Crisola Tan
Date:	10/3/26

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
    - A laptop or desktop computer
    - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
    - Google Drive for shared documentation
    - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases.
Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✔ Similarity score or ranking
✔ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✔ "New case saved" or similar confirmation
✔ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with “theft involving electronic transfer.” System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:
The reasoning engine adjusts argument patterns to match the new technical context.
Learning:
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.
Student submits a math solution with repeated errors.
System retrieves past students with similar error patterns (e.g., misusing formulas).
It adapts feedback and personalized lessons based on successful interventions.
Learning:
System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

```
# Step 3: Define Case Base
case_base = [
    {
        "student": "Student A",
        "error": "Wrong Formula",
        "topic": "Algebra",
           "solution": "Review algebra formulas and complete 5 practice
problems."
    },
    {
        "student": "Student B",
        "error": "Arithmetic Error",
        "topic": "Fractions",
         "solution": "Practice fraction operations and complete worksheet
exercises."
    },
    {
        "student": "Student C",
        "error": "Sign Error",
        "topic": "Integers",
        "solution": "Review positive and negative number rules."
    },
    {
```

```
        "student": "Student D",
        "error": "Wrong Formula",
        "topic": "Geometry",
        "solution": "Study geometry formulas and answer review questions."
    },
    {
        "student": "Student E",
        "error": "Calculation Error",
        "topic": "Decimals",
        "solution": "Complete decimal exercises and check work carefully."
    }
]

# Step 4: Similarity Assessment (Attribuite matching)
new_case = {
    "student": "Student F",
    "error": "Wrong Formula",
    "topic": "Algebra"
}

print("\nNEW PROBLEM")
print("-" * 30)
print(f"Student: {new_case['student']}")
print(f"Error: {new_case['error']}")
print(f"Topic: {new_case['topic']}")

best_case = None
highest_score = -1

print("\nSIMILARITY SCORES")
print("-" * 30)

for case in case_base:
    score = 0

    if case["error"] == new_case["error"]:
        score += 1
```

```
    if case["topic"] == new_case["topic"]:
        score += 1

    print(f"{case['student']}: {score}")

    if score > highest_score:
        highest_score = score
        best_case = case

print("\nMOST RELEVANT CASE")
print("-" * 30)
print(f"Student: {best_case['student']}")
print(f"Error: {best_case['error']}")
print(f"Topic: {best_case['topic']}")
print(f"Similarity Score: {highest_score}")


# Step 5: Adapt the Solution
adapted_solution = best_case["solution"]

if new_case["error"] == "Wrong Formula":
      adapted_solution += " Study worked examples and focus on formula
selection."

elif new_case["error"] == "Arithmetic Error":
     adapted_solution += " Practice calculation exercises and check each
step carefully."

elif new_case["error"] == "Sign Error":
     adapted_solution += " Review positive and negative number rules with
guided exercises."

print("\nADAPTED FINAL SOLUTION")
14
print("-" * 30)
15
```

```
print(adapted_solution)



# Step 6: Retain Stage
new_case["solution"] = adapted_solution
case_base.append(new_case)

print("\nCASE SAVED")
print("-" * 30)
print("New case saved successfully!")

print("\nUPDATED CASE BASE")
print("-" * 30)

for case in case_base:
    print(f"Student: {case['student']}")
    print(f"Error: {case['error']}")
    print(f"Topic: {case['topic']}")
    print(f"Solution: {case['solution']}")
    print()
```

Description of the Case Base

The case base consists of five previously recorded student learning cases. Each case contains information about a student's learning difficulty, including the type of error, the topic involved, and the instructional solution that was previously used to address the problem. These cases serve as the knowledge source for the Case-Based Reasoning (CBR) system. When a new student case is introduced, the system searches the case base to identify the most similar past case and uses its solution as the basis for generating a recommendation.

The attributes stored in each case are:

- Student Name
- Error Type
- Subject Topic
- Suggested Learning Solution

The stored cases include common mathematics learning difficulties such as wrong formula usage, arithmetic errors, sign errors, and calculation errors.

Explanation of Similarity Method

The system uses an attribute matching approach to determine the similarity between a new student case and the cases stored in the case base. Similarity is evaluated by comparing the student's error type and learning topic with those of previously recorded cases. A score is assigned based on the number of matching attributes, and the case with the highest similarity score is selected as the most relevant case. This method allows the system to efficiently identify past experiences that closely resemble the current learning problem and provides a basis for generating an appropriate recommendation.

Adaptation Logic

After retrieving the most relevant case, the system adapts the previously stored solution to better address the needs of the new student. Rather than directly copying the retrieved solution, the system modifies the recommendation according to the student's specific error type. For students who exhibit a wrong formula error, the system supplements the original intervention with additional guidance focused on reviewing worked examples and improving formula selection skills. This adaptation process enables the system to provide more personalized and targeted feedback while leveraging knowledge gained from past cases. Through this approach, the system demonstrates the reuse and revision stages of the Case-Based Reasoning cycle by applying previous experience to a new learning scenario.

Program Execution Screenshots

Figure 1. New Problem and Similarity Assessment

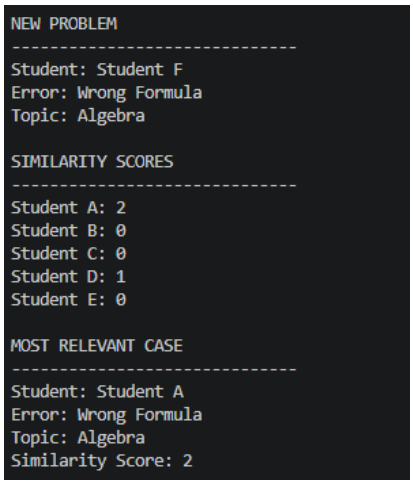


Figure 2. Adapted Final Solution

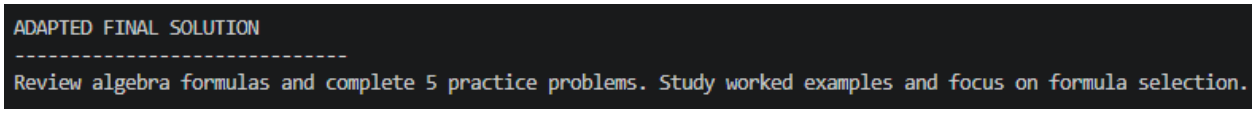


Figure 3. Updated Case Base

```
CASE SAVED
------------------------------
New case saved successfully!

UPDATED CASE BASE
------------------------------
Student: Student A
Error: Wrong Formula
Topic: Algebra
Solution: Review algebra formulas and complete 5 practice problems.

Student: Student B
Error: Arithmetic Error
Topic: Fractions
Solution: Practice fraction operations and complete worksheet exercises.

Student: Student C
Error: Sign Error
Topic: Integers
Solution: Review positive and negative number rules.

Student: Student D
Error: Wrong Formula
Topic: Geometry
Solution: Study geometry formulas and answer review questions.

Student: Student E
Error: Calculation Error
Topic: Decimals
Solution: Complete decimal exercises and check work carefully.

Student: Student F
Error: Wrong Formula
Topic: Algebra
Solution: Review algebra formulas and complete 5 practice problems. Study worked examples and focus on formula
selection.
```

In this activity, we worked together to develop a Case-Based Reasoning (CBR) system for an Intelligent Tutoring System. Magbanua was responsible for designing and implementing the Python code, including the case base, similarity assessment, solution adaptation, and retention processes. Panganiban focused on preparing the documentation, organizing the program outputs, and explaining the system's functionality. One challenge we encountered was determining how to implement the similarity assessment and adaptation stages in a way that accurately reflected the CBR cycle. We addressed this by using an attribute-matching approach and adapting solutions based on the student's specific error type. Through this activity, we learned how CBR applies past experiences to solve new problems through the retrieve, reuse, revise, and retain stages. Overall, the project improved our understanding of knowledge representation, reasoning, and the practical application of intelligent systems in education.